



프로젝트1- 웹 광고 클릭률 예측 AI 경진대회

OB 1조

김경민, 유재은, 전예원, 편한비

웹 광고 클릭률 예측 AI 경진대회

주제: 광고 클릭률을 예측하는 AI 모델 개발

설명: 7일간의 웹 로그를 기반으로 하루 동안의 광고 클릭률을 예측하는 AI 알고리즘 개발

데이터 구성:

- 시간 순으로 나열된 7일 동안의 웹 광고 클릭 로그
- ID: train 데이터 샘플 고유 ID
- Click: 예측 목표인 클릭 여부
- 0: 클릭하지 않음, 1: 클릭
- F01 ~ F39 : 각 클릭 로그와 연관된 Feature
- 개인정보 보호를 위해 상세 정보는 비식별 처리됨

ID	Click	F01	F02	F03	F04	F05
TRAIN_00000000	1	NSLHFNS	AVKQTCL	DTZFPRW	114.0	ISV
TRAIN_00000001	0	VGIVWZQ	LSUSMVO	PQGWFJZ	26.0	NF
TRAIN_00000002	0	JCDXFYU	PILDDJU	IAGJDOH	119.0	LFF
TRAIN_00000003	1	PSMFWTP	ZYAVJHP		15.0	ATK
TRAIN_00000004	0	SLCRICD	QPQWGX		13.0	CH
TRAIN_00000005	0	ZRTPTHN	WOAIOXV		10.0	KG
TRAIN_00000006	1	JCDXFYU	PILDDJU	IAGJDOH		LFF
TRAIN_00000007	0	KTNHUXF	YSUQLFP			FN
TRAIN_00000008	0	AOKIKZU	PKLWPDW		2.0	VR
TRAIN_00000009	0	JCDXFYU	PILDDJU	IAGJDOH	15.0	LFF

전처리

데이터 불러오기 및 불필요한 컬럼 드랍:

결측비율 0.3 이상인 컬럼, ID 컬럼 삭제

```
import pandas as pd
import numpy as np
```

```
train = pd.read_csv("C:\\Users\\LG\\Desktop\\활동\\ESAA\\프로젝트\\0B 프로젝트1\\train_01.csv")
test = pd.read_csv("C:\\Users\\LG\\Desktop\\활동\\ESAA\\프로젝트\\0B 프로젝트1\\open\\test_01.csv")
```

```
test = test.drop(columns=["F03", "F15", "F20", "F24", "F26", "F27", "F29"])
```

```
train = train.drop(columns=["ID"])
test = test.drop(columns=["ID"])
```

```
train.isna().mean().sort_values(ascending=False)
```

1.5s

F29	0.386523
F27	0.386523
F15	0.368684
F20	0.368684
F26	0.368684
F03	0.368684
F24	0.314059

F36	0.255563
F18	0.255563
F04	0.200698
F11	0.103415
F33	0.090260
F19	0.090260
F01	0.043238
F02	0.043238
F05	0.043238
F34	0.043238
F10	0.043238
F12	0.043238
F38	0.027894
F32	0.008721
F07	0.000000
F06	0.000000
Click	0.000000
ID	0.000000
...	
F31	0.000000
F35	0.000000
F37	0.000000
F39	0.000000
dtype:	float64

전처리

수치형 변수 빈칸 -999로 채우기

```
float_columns = ["F04", "F11", "F18", "F19", "F32", "F33", "F36", "F38"]
for col in float_columns:
    train[col] = train[col].fillna(-999)
    test[col] = test[col].fillna(-999)
```

범주형 변수 빈칸 missing으로 채우기

```
categorical_columns = ["F01", "F02", "F05", "F07", "F08", "F09", "F10", "F12", "F13", "F16", "F17", "F21", "F22", "F23", "F25", "F28", "F30", "F31", "F34", "F35", "F37", "F39"]
```

```
for col in categorical_columns:
    train[col] = train[col].fillna('missing')
    test[col] = test[col].fillna('missing')
```

전처리

고유값이 적은 변수 라벨 인코딩

```
from sklearn.preprocessing import LabelEncoder

encoders = {}
low_unique = ["F17", "F31", "F35"]

for col in low_unique:
    le = LabelEncoder()

    combined = list(train[col].astype(str)) + list(test[col].astype(str))
    le.fit(combined)

    train[col] = le.transform(train[col])
    test[col] = le.transform(test[col])

    encoders[col] = le
```

Frequency Encoding

Frequency Encoding: 범주형 변수의 각 값이 데이터에서 얼마나 자주 등장하는지(빈도)를 숫자로 변환하는 인코딩 방법

$$freq(x) = \frac{count(x)}{\text{전체 데이터 개수}}$$

```
middle_unique = ["F02", "F07", "F08", "F09", "F10", "F13", "F16", "F21", "F22", "F23", "F25", "F28", "F30", "F37", "F39"]
```

```
for col in middle_unique:
    freq = train[col].value_counts(normalize=True)

    train[col + '_freq'] = train[col].map(freq).fillna(0)
    test[col + '_freq'] = test[col].map(freq).fillna(0)
```

중간 수준의 고유값 개수를 가진 칼럼에만 적용

Frequency Encoding

Frequency Encoding: 범주형 변수의 각 값이 데이터에서 얼마나 자주 등장하는지(빈도)를 숫자로 변환하는 인코딩 방법

$$freq(x) = \frac{count(x)}{\text{전체 데이터 개수}}$$

```
middle_unique = ["F02", "F07", "F08", "F09", "F10", "F13", "F16", "F21", "F22", "F23", "F25", "F28", "F30", "F37", "F39"]

for col in middle_unique:
    freq = train[col].value_counts(normalize=True)
    train[col + '_freq'] = train[col].map(freq).fillna(0)
    test[col + '_freq'] = test[col].map(freq).fillna(0)
```

각 값의 비율(확률) 계산

Frequency Encoding

Frequency Encoding: 범주형 변수의 각 값이 데이터에서 얼마나 자주 등장하는지(빈도)를 숫자로 변환하는 인코딩 방법

$$freq(x) = \frac{count(x)}{\text{전체 데이터 개수}}$$

```
middle_unique = ["F02", "F07", "F08", "F09", "F10", "F13", "F16", "F21", "F22", "F23", "F25", "F28", "F30", "F37", "F39"]

for col in middle_unique:
    freq = train[col].value_counts(normalize=True)

    train[col + '_freq'] = train[col].map(freq).fillna(0)
    test[col + '_freq'] = test[col].map(freq).fillna(0)
```

각 카테고리를 해당 빈도 값으로 변환

Frequency Encoding

Frequency Encoding: 범주형 변수의 각 값이 데이터에서 얼마나 자주 등장하는지(빈도)를 숫자로 변환하는 인코딩 방법

$$freq(x) = \frac{count(x)}{\text{전체 데이터 개수}}$$

```
middle_unique = ["F02", "F07", "F08", "F09", "F10", "F13", "F16", "F21", "F22", "F23", "F25", "F28", "F30", "F37", "F39"]

for col in middle_unique:
    freq = train[col].value_counts(normalize=True)

    train[col + '_freq'] = train[col].map(freq).fillna(0)
    test[col + '_freq'] = test[col].map(freq).fillna(0)
```

test에 train에 없던 값 등장 시 -> 0으로 처리

Frequency Encoding

Frequency Encoding의 장점

- 차원 증가 없음 (One-hot 대비)
- 빠르고 간단
- 트리 모델(LGBM, CatBoost 등)에 잘 맞음
- 희소한 카테고리 자동으로 작은 값 부여

Frequency Encoding

Frequency Encoding의 기대효과

- 과적합 감소: 희귀 카테고리 -> 낮은 빈도로 자연스럽게 regularization 효과
- 정보 보존: 단순 label encoding보다 데이터 분포 정보 반영
- 모델 학습 안정화: 숫자형 feature로 변환 -> 모델이 더 잘 활용
- 차원 폭발 방지: one hot 대비 feature 수 증가 없음
- 일반화 성능 향상: 특히 test에서 unseen 값 -> 0 처리로 안정적 대응

Target Encoding

Target Encoding: 범주형 변수의 각 값을 타겟값의 평균으로 변환하는 인코딩 방법

$$TE(x) = E[y|x]$$

=> 특정 카테고리에서 타겟이 발생할 확률을 의미

```
from sklearn.model_selection import KFold
```

```
high_unique = ["F01", "F05", "F12", "F34"]
```

 고유값이 매우 많은 칼럼에만 적용

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
overall_mean = train['Click'].mean()
```

 전체 평균

Target Encoding

```
for col in high_unique:
    train[col + '_te'] = np.nan

    for train_idx, val_idx in kf.split(train):
        X_tr = train.iloc[train_idx]
        X_val = train.iloc[val_idx]

        target_mean = X_tr.groupby(col)['Click'].mean()

        train.loc[val_idx, col + '_te'] = X_val[col].map(target_mean)

# train의 NaN 채우기
train[col + '_te'] = train[col + '_te'].fillna(overall_mean)

# test는 전체 train 기준으로 인코딩
target_mean_full = train.groupby(col)['Click'].mean()
test[col + '_te'] = test[col].map(target_mean_full)

# test의 NaN도 채우기
test[col + '_te'] = test[col + '_te'].fillna(overall_mean)
```

KFold 기반 Target Encoding

Target Encoding

```
for col in high_unique:
    train[col + '_te'] = np.nan

    for train_idx, val_idx in kf.split(train):
        X_tr = train.iloc[train_idx]
        X_val = train.iloc[val_idx]

        target_mean = X_tr.groupby(col)['Click'].mean()

        train.loc[val_idx, col + '_te'] = X_val[col].map(target_mean)

# train의 NaN 채우기
train[col + '_te'] = train[col + '_te'].fillna(overall_mean)

# test는 전체 train 기준으로 인코딩
target_mean_full = train.groupby(col)['Click'].mean()
test[col + '_te'] = test[col].map(target_mean_full)

# test의 NaN도 채우기
test[col + '_te'] = test[col + '_te'].fillna(overall_mean)
```

unseen 카테고리 -> 전체 평균 사용

Target Encoding

```
for col in high_unique:
    train[col + '_te'] = np.nan

    for train_idx, val_idx in kf.split(train):
        X_tr = train.iloc[train_idx]
        X_val = train.iloc[val_idx]

        target_mean = X_tr.groupby(col)['Click'].mean()

        train.loc[val_idx, col + '_te'] = X_val[col].map(target_mean)

# train의 NaN 채우기
train[col + '_te'] = train[col + '_te'].fillna(overall_mean)

# test는 전체 train 기준으로 인코딩
target_mean_full = train.groupby(col)['Click'].mean()
test[col + '_te'] = test[col].map(target_mean_full)

# test의 NaN도 채우기
test[col + '_te'] = test[col + '_te'].fillna(overall_mean)
```

test는 train 전체 기준으로 적용

Target Encoding

Target Encoding의 장점

- 타겟 정보 직접 반영 -> 강력한 feature
- High cardinality에서도 효과적
- 차원 증가 없음
- 트리 모델에서 성능 크게 향상

Target Encoding

Target Encoding의 기대효과

- 성능 향상: 클릭 확률 값은 실제 타겟 패턴 반영
- 고차원 문제 해결: high cardinality -> 1개의 숫자로 압축
- 모델 학습 효율 증가: 불필요한 feature 제거, 더 빠르고 안정적인 학습
- 일반화 성능 향상: Kfold 적용으로 leakage 방지, 과적합 감소
- unseen 데이터 대응: 없는 카테고리 -> 전체 평균으로 안정 처리

Categorical columns drop

인코딩을 통해 새로운 feature 생성 (Label / Frequency / Target Encoding)

원본 범주형 변수는 정보 중복 및 노이즈 유발 가능

모델 성능 향상을 위해 원본 칼럼 제거

```
drop_columns = ["F01", "F02", "F05", "F07", "F08", "F09", "F10", "F12", "F13", "F16", "F21", "F22", "F23", "F25", "F28", "F30", "F34", "F37", "F39"]  
train = train.drop(columns=drop_columns)  
test = test.drop(columns=drop_columns)
```

후보1: XGB 단일 모델

```
# 1. 데이터 불러오기
X = train.drop(columns=['Click'])
y = train['Click']

# 2. 하이퍼파라미터 탐색 범위 설정
param_dist = {
    'n_estimators': [500, 800],          # 너무 많으면 오래 걸리므로 적당히
    'learning_rate': [0.03, 0.05, 0.1],  # 학습 속도
    'max_depth': [4, 6, 8],              # 트리 깊이 (너무 깊으면 과적합)
    'min_child_weight': [1, 3, 5],       # 자식 노드에 필요한 최소 가중치 할
    'subsample': [0.7, 0.8, 0.9],        # 데이터 샘플링
    'colsample_bytree': [0.7, 0.8, 0.9],  # 피쳐 샘플링
    'gamma': [0, 0.1, 0.2]              # 노드 분할을 위한 최소 손실 감소값
}

# 3. XGBoost 모델 초기화
xgb_model = xgb.XGBClassifier(
    tree_method='hist',
    device="cpu",
    random_state=42,
    eval_metric='auc'
)

# 4. RandomizedSearchCV 설정
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(
    estimator=xgb_model,
    param_distributions=param_dist,
    n_iter=10,          # 총 10번의 조합만 테스트
    scoring='roc_auc',  # 평가지표 AUC
    cv=cv,              # 교차 검증
    verbose=1,          # 진행 상황 출력
    n_jobs=1,           # 모든 CPU 코어 사용
    random_state=42)

# 5. 튜닝 시작
print("하이퍼파라미터 튜닝을 시작합니다... (약 수 분 소요)")
random_search.fit(X, y)

# 6. 최적의 결과 확인
print("\n[Best Parameters]:", random_search.best_params_)
print(f"[Best AUC Score]: {random_search.best_score_:.4f}")

# 7. 최적의 모델로 테스트 데이터 예측
best_model = random_search.best_estimator_
test_preds = best_model.predict_proba(test)[:, 1]

# 8. 제출 파일 저장
submission = pd.read_csv("C:/Users/User/OneDrive/바탕 화면/esaa_project1/sample_submission.csv")
submission['Click'] = test_preds
submission.to_csv("xgb_1.csv", index=False)
```



public점수
private점수

0.7571022109
0.7574767477

후보2: LGBM 단일 모델

```
def objective(trial):
    # [A] 튜닝용 샘플링
    sample_idx = np.random.choice(X_train.index, size=500000, replace=False)
    X_sample = X_train.loc[sample_idx]
    y_sample = y_train.loc[sample_idx]

    # [B] 탐색할 파라미터 범위
    param = {
        'objective': 'binary',
        'metric': 'auc',
        'boosting_type': 'gbdt',
        'random_state': 42,
        'is_unbalance': True, # 불균형 데이터 필수 옵션
        'verbosity': -1,

        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1),
        'num_leaves': trial.suggest_int('num_leaves', 31, 255),
        'max_depth': trial.suggest_int('max_depth', 7, 15),
        'min_child_samples': trial.suggest_int('min_child_samples', 20, 100),
        'feature_fraction': trial.suggest_float('feature_fraction', 0.6, 1.0),
        'bagging_fraction': trial.suggest_float('bagging_fraction', 0.6, 1.0),
        'bagging_freq': trial.suggest_int('bagging_freq', 1, 7),
        'lambda_l1': trial.suggest_float('lambda_l1', 1e-8, 10.0, log=True),
        'lambda_l2': trial.suggest_float('lambda_l2', 1e-8, 10.0, log=True),
    }

    # [C] 학습 (Early Stopping 적용)
    train_ds = lgb.Dataset(X_sample, label=y_sample)
    val_ds = lgb.Dataset(X_val, label=y_val, reference=train_ds)

    model = lgb.train(
        param,
        train_ds,
        num_boost_round=500,
        valid_sets=[val_ds],
        callbacks=[lgb.early_stopping(stopping_rounds=30)]
    )

    preds = model.predict(X_val)
    auc = roc_auc_score(y_val, preds)
    return auc
```

```
# 2. 최적화 실행
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20)

print('최적 파라미터:', study.best_params)

# [D] 최적 파라미터 적용
best_params = study.best_params
best_params['objective'] = 'binary'
best_params['metric'] = 'auc'
best_params['is_unbalance'] = True
best_params['verbosity'] = -1

# 최종 데이터셋
train_full_ds = lgb.Dataset(X_train, label=y_train, free_raw_data=True)
val_full_ds = lgb.Dataset(X_val, label=y_val, reference=train_full_ds, free_raw_data=True)

final_model = lgb.train(
    best_params,
    train_full_ds,
    num_boost_round=2000,
    valid_sets=[train_full_ds, val_full_ds],
    callbacks=[lgb.early_stopping(stopping_rounds=100), lgb.log_evaluation(period=100)]
)

from sklearn.metrics import classification_report, confusion_matrix

# 검증 데이터 평가
final_preds = final_model.predict(X_val)
final_auc = roc_auc_score(y_val, final_preds)
print(f"\n 최종 검증 AUC: {final_auc:.4f}")

# 테스트 데이터 예측
print("\n테스트 데이터 예측 중")
X_test_final = X_test_final.astype({col: 'float32' for col in X_test_final.select_dtypes('float64').columns})
X_test_final = X_test_final.astype({col: 'int32' for col in X_test_final.select_dtypes('int64').columns})

test_preds = final_model.predict(X_test_final)

# 3. 제출 파일 생성
sub = pd.read_csv('/content/drive/MyDrive/ESSA/sample_submission.csv')
sub['Click'] = test_preds
sub.to_csv('submission_optuna_lgbm.csv', index=False)
```

public점수
private점수

0.758873926
0.7590766408

후보3: CatBoost 단일 모델

```
# Pool로 묶으면서 범주형 변수(cat_features)를 바로 지정
train_pool = Pool(X_tr, y_tr, cat_features=categorical_columns)
val_pool = Pool(X_va, y_va, cat_features=categorical_columns)
test_pool = Pool(test_X, cat_features=categorical_columns)

# Pool을 만들었으니 판다스 데이터프레임들도 다 날리기
del X_tr, y_tr, X_va, y_va, test_X
gc.collect()

model = CatBoostClassifier(
    iterations=1000,
    learning_rate=0.05,
    eval_metric='AUC',
    random_seed=42,
    verbose=100,
    early_stopping_rounds=50,
    max_ctr_complexity=1)

# Pool 객체를 그대로
model.fit(train_pool,
        eval_set=val_pool,
        use_best_model=True)

# 5. 예측 및 결과 확인
val_preds = model.predict(val_pool, prediction_type='Probability')[:, 1]
# val_pool.get_label()로 정답을 가져와서 비교합니다
val_auc = roc_auc_score(val_pool.get_label(), val_preds)
print(f"\nCatBoost 검증 세트 AUC Score: {val_auc:.5f}")
```

```
# 6. 제출 파일 만들기
test_preds = model.predict(test_pool, prediction_type='Probability')[:, 1]

zip_path = "/content/drive/MyDrive/ESAA/OB/project1/open.zip"
with zipfile.ZipFile(zip_path, "r") as z:
    with z.open("sample_submission.csv") as f:
        submission = pd.read_csv(f)

submission['Click'] = test_preds
submission_path = "/content/drive/MyDrive/ESAA/OB/project1/catboost_native_optimized.csv"
submission.to_csv(submission_path, index=False)
```



public점수
private점수

0.758548782
0.7586707783

최종모델: CatBoost

- CatBoost에는 categorical 컬럼에 인코딩을 적용하지 않음

```
# 전처리 완료한 CatBoost 전용 데이터 로드(인코딩 X)
train_cat = pd.read_csv('/content/drive/MyDrive/ESAA/OB/project1/train_catboost1.csv')
test_cat = pd.read_csv('/content/drive/MyDrive/ESAA/OB/project1/test_catboost1.csv')

X_cat = train_cat.drop(columns=['Click'])
y = train_cat['Click']
categorical_columns = ["F01", "F02", "F05", "F07", "F08", "F09", "F10", "F12", "F13", "F16", "F17",
                        "F21", "F22", "F23", "F25", "F28", "F30", "F31", "F34", "F35", "F37", "F39"]
```

최종모델: CatBoost

[CatBoost 사용시, categorical 컬럼에 인코딩을 적용하지 않는 이유]

타겟 인코딩은 범주별로
타겟값의 평균을 냄

-> 모델이 정답을 미리 보고
학습하는 **데이터 누수**와 **과적합**
문제가
일어날 수 있음

중요한 정보를 담고 있을 수 있는
범주형 변수의 모든 조합을
미리 인코딩하면,
변수가 기하급수적으로 늘어남



데이터를 시간에 따라 들어온 것처럼
가상으로 순서를 섞고,
현재 데이터 이전의 값들만을
사용하여 통계량을 계산함

트리를 분할할 때마다
현재 분할에 사용된 범주형 변수와
데이터셋의 다른 범주형 변수를
조합함

-> 유의미한 상호작용을 스스로
찾아내어 인코딩에 반영함

최종모델: CatBoost + LGBM 스택킹

- CatBoost

```
# 데이터 분할 및 모델 정의
X_tr_cat, X_va_cat, y_tr, y_va = train_test_split(X_cat, y, test_size=0.2, stratify=y, random_state=42)

# 하이퍼파라미터 및 학습 설정 (학습 단계)
model_cat = CatBoostClassifier(iterations=1000, learning_rate=0.05, cat_features=categorical_columns,
                               eval_metric='AUC', random_seed=42, verbose=False)
model_cat.fit(X_tr_cat, y_tr, eval_set=(X_va_cat, y_va), early_stopping_rounds=50)

# 예측 및 성능 평가
cat_val_preds = model_cat.predict_proba(X_va_cat)[: , 1]
cat_test_preds = model_cat.predict_proba(test_cat)[: , 1]

print(f"CatBoost 검증 AUC: {roc_auc_score(y_va, cat_val_preds):.5f}")

# 램 확보를 위해 CatBoost 데이터 삭제
del train_cat, test_cat, X_cat, X_tr_cat, X_va_cat, model_cat
gc.collect()
```

최종모델: CatBoost + LGBM 스택킹

- CatBoost

```
# 데이터 분할 및 모델 정의
X_tr_cat, X_va_cat, y_tr, y_va = train_test_split(X_cat, y, test_size=0.2, stratify=y, random_state=42)

# 하이퍼파라미터 및 학습 설정 (학습 단계)
model_cat = CatBoostClassifier(iterations=1000, learning_rate=0.05, cat_features=categorical_columns,
                               eval_metric='AUC', random_seed=42, verbose=False)
model_cat.fit(X_tr_cat, y_tr, eval_set=(X_va_cat, y_va), early_stopping_rounds=50)

# 예측 및 성능 평가
cat_val_preds = model_cat.predict_proba(X_va_cat)[: , 1]
cat_test_preds = model_cat.predict_proba(test_cat)[: , 1]

print(f"CatBoost 검증 AUC: {roc_auc_score(y_va, cat_val_preds):.5f}")

# 램 확보를 위해 CatBoost 데이터 삭제
del train_cat, test_cat, X_cat, X_tr_cat, X_va_cat, model_cat
gc.collect()
```

최종모델: CatBoost + LGBM 스택킹

- CatBoost

```
# 데이터 분할 및 모델 정의
X_tr_cat, X_va_cat, y_tr, y_va = train_test_split(X_cat, y, test_size=0.2, stratify=y, random_state=42)

# 하이퍼파라미터 및 학습 설정 (학습 단계)
model_cat = CatBoostClassifier(iterations=1000, learning_rate=0.05, cat_features=categorical_columns,
                               eval_metric='AUC', random_seed=42, verbose=False)
model_cat.fit(X_tr_cat, y_tr, eval_set=(X_va_cat, y_va), early_stopping_rounds=50)

# 예측 및 성능 평가
cat_val_preds = model_cat.predict_proba(X_va_cat)[: , 1]
cat_test_preds = model_cat.predict_proba(test_cat)[: , 1]

print(f"CatBoost 검증 AUC: {roc_auc_score(y_va, cat_val_preds):.5f}")

# 램 확보를 위해 CatBoost 데이터 삭제
del train_cat, test_cat, X_cat, X_tr_cat, X_va_cat, model_cat
gc.collect()
```

CatBoost 검증 AUC: 0.76398

최종모델: CatBoost + LGBM 스택킹

- CatBoost

```
# 데이터 분할 및 모델 정의
X_tr_cat, X_va_cat, y_tr, y_va = train_test_split(X_cat, y, test_size=0.2, stratify=y, random_state=42)

# 하이퍼파라미터 및 학습 설정 (학습 단계)
model_cat = CatBoostClassifier(iterations=1000, learning_rate=0.05, cat_features=categorical_columns,
                               eval_metric='AUC', random_seed=42, verbose=False)
model_cat.fit(X_tr_cat, y_tr, eval_set=(X_va_cat, y_va), early_stopping_rounds=50)

# 예측 및 성능 평가
cat_val_preds = model_cat.predict_proba(X_va_cat)[: , 1]
cat_test_preds = model_cat.predict_proba(test_cat)[: , 1]

print(f"CatBoost 검증 AUC: {roc_auc_score(y_va, cat_val_preds):.5f}")

# 램 확보를 위해 CatBoost 데이터 삭제
del train_cat, test_cat, X_cat, X_tr_cat, X_va_cat, model_cat
gc.collect()
```

최종모델: CatBoost + LGBM 스택킹

```
# LGBM 전용 수치형 데이터 로드
train_lgbm = pd.read_csv('/content/drive/MyDrive/ESAA/OB/project1/train_final.csv')
test_lgbm = pd.read_csv('/content/drive/MyDrive/ESAA/OB/project1/test_final.csv')

X_lgbm = train_lgbm.drop(columns=['Click'])

X_tr_lgbm, X_va_lgbm, _, _ = train_test_split(X_lgbm, y, test_size=0.2, stratify=y, random_state=42)

# 찾아낸 최적의 파라미터 딕셔너리 정의
lgbm_best_params = {
    'learning_rate': 0.04423786044029868,
    'num_leaves': 155,
    'max_depth': 13,
    'min_child_samples': 76,
    'feature_fraction': 0.6700322150966446,
    'bagging_fraction': 0.883453460183011,
    'bagging_freq': 4,
    'lambda_l1': 3.342536329323218e-05,
    'lambda_l2': 3.1845692568661615e-07,
    'n_estimators': 2000,
    'random_state': 42,
    'n_jobs': -1 }

# 모델 생성
model_lgbm = LGBMClassifier(**lgbm_best_params)
```

최종모델: CatBoost + LGBM 스택킹

- LGBM

```
# LGBM 전용 수치형 데이터 로드
train_lgbm = pd.read_csv('/content/drive/MyDrive/ESAA/OB/project1/train_final.csv')
test_lgbm = pd.read_csv('/content/drive/MyDrive/ESAA/OB/project1/test_final.csv')

X_lgbm = train_lgbm.drop(columns=['Click'])

X_tr_lgbm, X_va_lgbm, _, _ = train_test_split(X_lgbm, y, test_size=0.2, stratify=y, random_state=42)

# 찾아낸 최적의 파라미터 딕셔너리 정의
lgbm_best_params = {
    'learning_rate': 0.04423786044029868,
    'num_leaves': 155,
    'max_depth': 13,
    'min_child_samples': 76,
    'feature_fraction': 0.6700322150966446,
    'bagging_fraction': 0.883453460183011,
    'bagging_freq': 4,
    'lambda_l1': 3.342536329323218e-05,
    'lambda_l2': 3.1845692568661615e-07,
    'n_estimators': 2000,
    'random_state': 42,
    'n_jobs': -1 }

# 모델 생성
model_lgbm = LGBMClassifier(**lgbm_best_params)
```

최종모델: CatBoost + LGBM 스택킹

- LGBM

```
# 학습 시작 (조기 종료 설정 추가)
from lightgbm import early_stopping, log_evaluation

model_lgbm.fit(X_tr_lgbm, y_tr,
               eval_set=[(X_va_lgbm, y_va)],
               callbacks=[early_stopping(stopping_rounds=100), log_evaluation(100)])

# 5. 예측값 뽑기
lgbm_val_preds = model_lgbm.predict_proba(X_va_lgbm)[: , 1]
lgbm_test_preds = model_lgbm.predict_proba(test_lgbm)[: , 1]

print(f"최적화된 LGBM 검증 AUC: {roc_auc_score(y_va, lgbm_val_preds):.5f}")

# 램 확보
del train_lgbm, test_lgbm, X_lgbm, X_tr_lgbm, X_va_lgbm, model_lgbm
import gc
gc.collect()
```

최종모델: CatBoost + LGBM 스택킹

- LGBM

```
# 학습 시작 (조기 종료 설정 추가)
from lightgbm import early_stopping, log_evaluation

model_lgbm.fit(X_tr_lgbm, y_tr,
               eval_set=[(X_va_lgbm, y_va)],
               callbacks=[early_stopping(stopping_rounds=100), log_evaluation(100)])

# 5. 예측값 뽑기
lgbm_val_preds = model_lgbm.predict_proba(X_va_lgbm)[: , 1]
lgbm_test_preds = model_lgbm.predict_proba(test_lgbm)[: , 1]

print(f"최적화된 LGBM 검증 AUC: {roc_auc_score(y_va, lgbm_val_preds):.5f}")

# 램 확보
del train_lgbm, test_lgbm, X_lgbm, X_tr_lgbm, X_va_lgbm, model_lgbm
import gc
gc.collect()
```

→ 최적화된 LGBM 검증 AUC: 0.75881

최종모델: CatBoost + LGBM 스택킹

- LGBM

```
# 학습 시작 (조기 종료 설정 추가)
from lightgbm import early_stopping, log_evaluation

model_lgbm.fit(X_tr_lgbm, y_tr,
               eval_set=[(X_va_lgbm, y_va)],
               callbacks=[early_stopping(stopping_rounds=100), log_evaluation(100)])

# 5. 예측값 뽑기
lgbm_val_preds = model_lgbm.predict_proba(X_va_lgbm)[: , 1]
lgbm_test_preds = model_lgbm.predict_proba(test_lgbm)[: , 1]

print(f"최적화된 LGBM 검증 AUC: {roc_auc_score(y_va, lgbm_val_preds):.5f}")

# 램 확보
del train_lgbm, test_lgbm, X_lgbm, X_tr_lgbm, X_va_lgbm, model_lgbm
import gc
gc.collect()
```

최종모델: CatBoost + LGBM 스택킹

```
# Base 모델들의 예측값(20% 검증셋)을 모아 새로운 문제지
stacked_train = pd.DataFrame({
    'CatBoost': cat_val_preds,
    'LGBM': lgbm_val_preds
})

# Base 모델들의 Test 예측값을 모아서 최종 시험지.
stacked_test = pd.DataFrame({
    'CatBoost': cat_test_preds,
    'LGBM': lgbm_test_preds
})
```

- stacked_train : 1단계 모델이 각각 20%의 검증(validation)데이터에 대해 내놓은 클릭 확률
- stacked_test : 각 모델이 전체 테스트 데이터를 보고 예측한 값

최종모델: CatBoost + LGBM 스택킹

```
# 선형 회귀 모델이 두 모델의 주장을 듣고 어떤 비율로 섞을지 학습
meta_model = LogisticRegression()
meta_model.fit(stacked_train, y_va)

# 스택킹 최종 성능 확인
meta_val_preds = meta_model.predict_proba(stacked_train)[: , 1]
meta_auc = roc_auc_score(y_va, meta_val_preds)
print(f"최종 스택킹 검증 AUC Score: {meta_auc:.5f}")
print(f"반영 비율 - CatBoost: {meta_model.coef_[0][0]:.4f} / LGBM: {meta_model.coef_[0][1]:.4f}")
```

- Meta Model - LogisticRegression 선택, 선형적인 관계를 학습하는 단순한 모델로 과적합 방지, 각 모델에 가중치 부여
- catboost 모델 AUC 값 : 0.7639
- LGBM 모델 AUC 값 : 0.7588
- 반영 비율 : catboost 3.88 / LGBM 2.20
- 최종 스택킹 모델 AUC 값 : **0.7659**

최종 저장 및 리더보드 제출

```
# 4. 최종 Test 예측 및 저장
final_preds = meta_model.predict_proba(stacked_test)[: , 1]

import zipfile
zip_path = "/content/drive/MyDrive/ESAA/OB/project1/open.zip"
with zipfile.ZipFile(zip_path, "r") as z:
    with z.open("sample_submission.csv") as f:
        submission = pd.read_csv(f)

submission['Click'] = final_preds
submission_path = "/content/drive/MyDrive/ESAA/OB/project1/stacking_fast_baseline.csv"
submission.to_csv(submission_path, index=False)
```

public점수
private점수
0.7659939527
0.7660740479

- 세션 저장 공간을 아끼기 위해 zip파일로 데이터 저장
- 리더보드 점수 : 약 0.7659, 모델링 결과와 비슷하여 과적합이 심하지 않았다는 것을 알 수 있음

결론 및 시사점

- 결론

- catboost 모델로 범주형 변수 간의 비선형적 조합을 찾고 LightGBM 모델은 타겟 인코딩을 통해 얻은 수치적 변별력을 학습했음 -> 스택킹 효과 커짐
- 최종 0.7659의 AUC 기록 -> 실제 CTR 예측 환경에서 클릭 가능성이 높은 사용자를 안정적으로 분류해낼 수 있는 수준

- 시사점

- 큰 데이터 용량으로 모델링 진행 시 많은 시간과 램이 소모되어 다양한 튜닝과 모델링 기법을 적용해보지 못해 아쉬움